



Project Report

Smart Task Manager API

Name: Anshika Singh

Registration no.: 25BCE10770

Branch: B.tech CSE Core

Year: Semester 2 (1 st Year)

Date: 4/05/2026

Project Overview

Smart Task Manager API is a backend task management system developed using Python. The project is designed to help users create, manage, and update daily tasks efficiently. It demonstrates backend development concepts such as API creation, routing, database integration, and data persistence.

This project uses FastAPI for building APIs, SQLAlchemy for database operations, and SQLite as the database engine for storing task information permanently.

The application provides a simple yet effective task management system where users can:

- Add new tasks
- View all tasks
- Mark tasks as completed
- Store tasks permanently in a database

This project serves as a foundational backend application for learning real-world software development using Python.

Problem Statement

Managing daily tasks manually can often become disorganized and inefficient. People frequently forget tasks, lose track of progress, or struggle to monitor completion status.

A digital task management backend solves this issue by allowing tasks to be stored, accessed, and updated systematically.

The Smart Task Manager API was built to provide a structured backend system for handling such task operations efficient

Objectives

The main objectives of this project are:

- To learn backend development using Python
- To understand REST API development
- To perform Create, Read, and Update operations
- To integrate databases with backend systems
- To understand ORM concepts
- To create scalable backend architecture
- To build a project suitable for GitHub portfolio

Technology Stack

Programming Language

Python

Backend Framework

FastAPI

Database

SQLite

ORM

SQLAlchemy

Development Environment

Visual studio code

API Testing

FastAPI Swagger Documentation Interface

Version Control

GitHub

Project Architecture

The project follows a modular backend architecture:

Client Request



API Endpoint Handling (FastAPI)



Business Logic Processing



Database Communication (SQLAlchemy ORM)



SQLite Database Storage



Response Returned to Client

Project Folder Structure

```
task-manager/  
| — venv/  
| — main.py  
| — database.py  
| — models/  
|   └─ models.py  
| — tasks.db  
| — requirements.txt  
| — README.md
```

Detailed Explanation of Files

main.py

This is the main application file responsible for:

- Starting FastAPI server
- Defining API routes
- Receiving requests
- Sending responses
- Connecting application logic with database

Main routes:

GET /

Checks whether API server is working.

Response:

```
{  
  "message": "API is working 🚀"  
}
```


POST /tasks

Creates a new task.

Input:

Title of task

Example:

Learn Python

Output:

New task stored in database

GET /tasks

Fetches all tasks stored in database.

Returns:

- Task ID
- Task Title
- Completion Status

PUT /tasks/{task_id}

Updates task completion status.

Changes:

False → True

database.py

This file manages database connection.

Functions:

- Creates database engine
- Creates sessions
- Defines base model class

Database file:

tasks.db

models/models.py

This file defines database table structure.

Task table contains:

id

Unique task identifier

title

Task name

completed

Boolean field:

- False → pending
- True → completed

Database Design

Table Name: tasks

Fields:

1.id → Integer → Primary Key

- 2.title → String
- 3.completed → Boolean

Features Implemented

- 1.Create Task
- 2.View Tasks
- 3.Update Task Status
- 4.Permanent Database Storage
- 5.API Testing Interface
- 6.Modular Code Structure
- 7.Lightweight Database Integration

API Endpoints

- 1.GET /
Check API status
- 2.POST /tasks
Create task
- 3.GET /tasks
Fetch all tasks
- 4.PUT /tasks/{task_id}
Mark task complete

Learning Outcomes

Through this project, the following concepts were learned:

- Python backend development
- REST API design
- FastAPI framework usage
- Database connectivity
- SQLAlchemy ORM
- CRUD operations
- Project structuring
- Debugging and testing APIs

Advantages

- Fast performance
- Simple architecture
- Easy to understand
- Beginner friendly
- Real-world backend concepts
- Scalable for future features

Future Enhancements

Possible future upgrades:

- Delete task feature
- User authentication
- Login/Signup system
- Task categories
- Due dates
- Priority levels
- Notifications
- Frontend interface
- Cloud deployment

Conclusion

Smart Task Manager API is a backend project that demonstrates how Python can be used to build efficient and scalable web APIs. By integrating FastAPI, SQLAlchemy, and SQLite, the project successfully performs task management operations while maintaining data permanently.

This project provides strong foundational knowledge in backend software development and serves as an excellent portfolio project for GitHub.